CityPulse: An Intelligent Parking Availability Platform

Vision-Based Slot Occupancy Detection for Urban Bangladesh

Final Report — Senior Design I

CSE499A — Section 15 — Project Group 03

Md. Rahad Hossain
ID: 2211316642

Md. Yousuf
ID: 2211461642

MD. Rokib Hasan Oli
ID: 2211950642

Department of Electrical and Computer Engineering
North South University, Dhaka, Bangladesh

***Abstract*—Unorganized roadside parking is a significant contributor to traffic congestion in Dhaka, where average travel speeds have fallen to approximately 7 km/h. Drivers cannot determine parking availability before arrival, and enforcement depends on manual deployment of traffic police that cannot scale city-wide. This paper presents CityPulse, a platform coupling vision-based slot occupancy detection with a multi-role service architecture connecting drivers, parking space owners, and traffic authorities. The occupancy model is a MobileNetV2 classifier trained in two phases on a merged corpus of 808,991 labelled slot images from the PKLot and CNRPark+EXT benchmarks, evaluated under a cross-lot protocol in which validation is drawn entirely from a parking lot absent from training — a stricter generalization test than the date-based splits commonly reported. The head-only phase reaches 97.28% accuracy and full fine-tuning raises this to 99.18%, an improvement of 1.90 percentage points on unseen infrastructure. A second model, VPS-Net, is reproduced to 98.53% on ps2.0 for automated slot geometry extraction. The trained model is served through a FastAPI service and consumed by a NestJS backend that reconciles visual predictions with active reservation state, so a slot held by a booking is never advertised as free. We report the training protocol, measured results, integration architecture, and an explicit account of present limitations.**

***Index Terms*—smart parking, parking occupancy detection, transfer learning, convolutional neural networks, cross-lot generalization, intelligent transportation systems, urban mobility, Bangladesh**

I. Introduction

Urban traffic congestion is a growing crisis in Bangladesh, particularly in Dhaka, one of the most densely populated cities in the world. A World Bank assessment reports that average traffic speed in Dhaka has declined to approximately 7 km/h, placing it among the slowest cities for road travel globally [1]. A substantial contributor to this decline is unorganized roadside parking, in which vehicles halt wherever space permits, obstructing lanes and initiating chain congestion that can persist for hours.

The conventional response, deploying traffic police at major intersections, does not scale: congestion points across a city the size of Dhaka far exceed available manpower, and officers must work extended hours in extreme heat, monsoon rain, and heavy pollution. Concurrently, Bangladesh has over 180 million mobile subscribers as of 2024 with a rising smartphone share [2], establishing a viable delivery channel for digital interventions in urban mobility.

CityPulse addresses this opportunity. Its central technical claim is that parking occupancy can be determined automatically from imagery captured by cameras already installed at parking facilities, removing the dependence on manual availability updates that limits every parking application currently deployed in Bangladesh. This report documents the machine learning methodology and measured results, the backend services that operationalize the model, the client applications that consume it, and a candid assessment of what the prototype does and does not yet do.

Sections V and VI present the AI/ML methodology and results, which constitute the core technical contribution; Sections VII through IX describe the platform that operationalizes them, and Section X states the system's boundaries.

II. Problem Statement

The problems CityPulse is designed to address are the following.

- **Unorganized roadside parking.** Drivers park on roadsides because organized alternatives are unknown to them at the moment of decision, producing lane blockage and congestion.
- **Absence of real-time availability information.** No deployed system in Bangladesh reports availability before arrival. Drivers circulate searching for space, and this search traffic itself contributes to congestion.
- **Manual availability reporting does not scale.** Existing local platforms depend on an operator updating a count by hand. Update latency grows with facility count, and accuracy degrades precisely when demand is highest.
- **Underutilized private capacity.** Private facilities hold unused capacity while adjacent roads carry illegal parking, because that capacity is not discoverable.
- **Unsustainable manual enforcement.** Stationing traffic police at every congested point is expensive, physically demanding, and impossible to scale city-wide.

III. Related Work

A. Vision-Based Occupancy Detection

Parking occupancy detection from fixed cameras is a mature research area. The PKLot dataset [3] established the standard benchmark, contributing 12,417 images and 695,899 segmented parking space patches captured across three Brazilian lots over more than thirty days under sunny, overcast, and rainy conditions. CNRPark+EXT [4] extended the problem to a multi-camera setting with nine distinct viewpoints over 164 spaces, and demonstrated that decentralized inference on embedded devices is feasible.

Two families of approach dominate. The first treats occupancy as per-slot binary classification, cropping each slot region with a fixed annotated polygon; the second treats it as object detection over the full frame. Classification is computationally cheaper and better suited to fixed cameras with stable geometry, while detection tolerates camera movement at higher cost. Recent work applying YOLOv8 backbone variants to PKLot reports mAP50 above 0.99 [7], while lightweight classifiers such as improved MobileNetV3 reach comparable accuracy at a fraction of the compute [8].

B. Parking Platforms

Commercial platforms such as ParkWhiz allow users to locate and reserve parking through mobile applications [13], but rely on operator-supplied availability rather than automated sensing.

Within Bangladesh, Dhaka North City Corporation launched a smart parking application in 2023 covering 202 on-street spaces in Gulshan [14], and Parking Koi operates a marketplace connecting drivers with private space owners [15]. Neither performs automated detection, integrates camera infrastructure, or serves all three stakeholder roles: both depend on manual availability updates over limited geography. That combination — automated visual detection over existing cameras, delivered to drivers, owners, and authorities in one system — is the gap CityPulse addresses.

C. The Bangladeshi Baseline

The work most directly relevant to this project is that of Prova et al. [9], also conducted at North South University. The authors train a binary CNN occupancy classifier on PKLot and CNRPark+EXT, then evaluate it against CCTV footage they collected themselves from Dhaka apartment parking facilities. Without any Bangladesh-specific training data, the model attains 86.1% accuracy on indoor and 90.5% on outdoor Dhaka scenes, and is confirmed operable on a Raspberry Pi 4.

Three findings from that work directly shape the design adopted here. First, the transfer result establishes that international benchmarks are a viable pretraining source for a Bangladeshi deployment, which justifies building on PKLot and CNRPark+EXT rather than waiting for local data. Second, the confirmed Raspberry Pi feasibility fixes the computational budget within which an architecture must be chosen. Third, the authors identify two open problems: they do not explore fine-tuning on Dhaka data, and they note that manual per-camera mask definition is the principal scalability bottleneck, since every new facility requires a human to define slot boundaries before classification can begin. Section V addresses the first through a two-phase fine-tuning protocol and a training corpus an order of magnitude larger, and the second through the slot detection component of Section V-F.

Related surveillance capabilities relevant to the platform's longer-term direction include Bangla script license plate recognition, for which public datasets remain scarce [10], [11], and traffic congestion estimation for Dhaka's heterogeneous traffic [12]. These are surveyed as groundwork for subsequent phases and are not implemented in the present system.

IV. Objectives

The project set out to train and validate an occupancy classifier generalizing to camera viewpoints and facilities unseen during training; serve the trained models through a documented inference API; implement a backend reconciling model predictions with reservation state into a single authoritative availability figure; deliver a driver mobile application providing location-based search over live backend data alongside a role-separated web portal for owners and authorities; establish a maintainable service-oriented architecture with documented contracts and reproducible training artifacts; and survey Bangla license plate recognition and congestion detection as groundwork for subsequent phases.

V. AI/ML Methodology

A. Task Formulation

Occupancy detection is formulated as per-slot binary image classification. Given a cropped image region corresponding to a single parking slot, the model outputs a label in $\{empty, occupied\}$ together with a softmax confidence score.

The occupancy pipeline follows the approach established by Prova et al. [9] and described in Section III-C: a mask defined per camera crops the full frame into one patch per slot, each patch is resized to 224×224 RGB and labelled zero for unoccupied and one for occupied, and the classifier is pretrained on PKLot and CNRPark+EXT before deployment to Bangladeshi footage. This project extends that baseline in three respects. The training corpus is enlarged from the 69,058-image subset used there to 808,991 images. Validation is moved from a same-lot to a cross-lot protocol. And the fine-tuning stage that the original authors identified as unexplored is implemented as an explicit second training phase.

Classification is chosen deliberately over full-frame object detection. The target deployment consists of cameras fixed in position at a registered facility, so each slot occupies a stable image region that can be annotated once at registration and reused indefinitely. Under this constraint, a detector's localization capability provides no benefit while imposing substantially higher computational cost. The intended edge target is a Raspberry Pi 4 or 5 at the parking site — the hardware class on which [9] demonstrated feasibility — which makes inference cost a primary design constraint rather than a secondary concern.

B. Datasets

Two public benchmarks were merged into a single classification corpus. Table I summarizes their properties.

TABLE I
SOURCE DATASETS

Dataset	Patches	Cameras	Conditions
PKLot [3]	695,851	3	sunny, overcast, rainy
CNRPark+EXT [4]	113,140	9	sunny, overcast, rainy, foggy
Merged	**808,991**	**12**	—

Merging is motivated by viewpoint diversity. PKLot contributes volume and weather variation but only three elevated angles, all Brazilian; CNRPark+EXT adds nine Italian viewpoints. A model trained on the union sees twelve distinct camera geometries rather than three — directly relevant to a deployment where every registered facility presents a new and uncontrolled angle.

Counts in Table I are those present in the merged corpus, marginally below the published totals cited in Section III. The corpus is organized into class-labelled directories and published for reproducibility.[1] Its composition is given in Table II.

TABLE II
MERGED DATASET COMPOSITION

Split	Occupied	Empty	Total
Train	287,970	336,589	624,559
Validation	110,841	73,591	184,432
Total	**398,811**	**410,180**	**808,991**

The class distribution is near balanced overall (49.3% occupied), so no resampling or class weighting was applied.

C. Evaluation Protocol

The validation split is drawn entirely from UFPR05, a lot contributing no images to training. This departs deliberately from the date-based splitting common in the occupancy literature, where training and validation frames come from the same camera on different days.

The distinction is material. Under a date-based split a model may score highly by memorizing each slot's fixed background and detecting deviation from it — a strategy that fails the moment the camera changes. Under the cross-lot protocol the validation lot presents unseen pavement, lighting geometry, slot markings, and camera elevation, so reported accuracy estimates performance on a newly registered facility: the operationally relevant quantity here.

[1] https://www.kaggle.com/datasets/raahad/parking-occupancy-merged

D. Model Architecture

The classifier is MobileNetV2 [5] with a replaced two-class output head:

$$\begin{aligned}&\text{input } (224 \times 224 \times 3)\\ &\quad \to \text{MobileNetV2 feature extractor}\\ &\quad \to \text{AdaptiveAvgPool2d}\\ &\quad \to \text{Dropout}(p{=}0.2)\\ &\quad \to \text{Linear}(1280 \to 2)\\ &\quad \to \text{Softmax} \to \arg\max \to \{\text{empty, occupied}\}\end{aligned}$$

Inputs are normalized with ImageNet statistics, mean $[0.485, 0.456, 0.406]$ and standard deviation $[0.229, 0.224, 0.225]$.

MobileNetV2 was selected for two reasons. First, transfer learning: the backbone arrives with low-level visual features already learned from 1.28 million ImageNet images, so training need only learn the empty/occupied distinction on top of an existing representation. Published results using custom CNNs trained from scratch on this dataset cluster near 93% accuracy, whereas pretrained MobileNet variants consistently reach 97–98%; the gap is attributable to transfer learning rather than architectural capacity. Second, inference cost: MobileNetV2's depthwise separable convolutions were designed for mobile and embedded execution, which matches the Raspberry Pi deployment target.

E. Two-Phase Training Protocol

Training proceeds in two phases, each executed as a separate notebook on a single NVIDIA Tesla T4 GPU. Hyperparameters are given in Table III.

TABLE III
TRAINING CONFIGURATION

Setting	Phase 1	Phase 2
Trainable layers	head only	all
Batch size	128	128
Learning rate	1×10^{-3}	1×10^{-5}
Epochs	5	10
Optimizer	Adam	Adam
Mixed precision	—	float16 (AMP)
Grad. checkpointing	—	2-segment split

Phase 1 freezes all MobileNetV2 layers and trains only the newly initialized two-class head. This brings the randomly initialized classifier to a stable operating point before any gradient reaches the pretrained backbone. Updating the backbone against a random head would otherwise destroy the ImageNet features that motivate the architecture choice.

Phase 2 unfreezes the full network and continues at a learning rate two orders of magnitude lower, permitting the backbone to adapt to parking imagery without overwriting its general visual representation. Automatic mixed precision at float16 and two-segment gradient checkpointing were required to hold the 128-image batch within available GPU memory.

F. Slot Detection: VPS-Net

The manual per-camera mask definition identified in [9] as the principal scalability bottleneck is addressed by a second model. VPS-Net [6] detects parking slot geometry directly from imagery through a two-stage pipeline: a YOLOv3 detector locates candidate slot marking points, and a customized AlexNet variant classifies each recovered slot as vacant or occupied. If slot geometry can be recovered automatically, facility onboarding no longer requires a human to trace slot boundaries.

The occupancy classification stage was reproduced and retrained on the ps2.0 around-view dataset. Of 15,754 annotated instances, 9,827 training images yield 8,948 slot samples (6,324 vacant, 2,624 non-vacant), doubled to 17,896 by the 180° rotation augmentation the original work prescribes; the held-out split yields 2,173 samples. Following the original label convention, instances marked *vehicle-in-slot* are excluded from training and mapped to non-vacant at evaluation. The reproduced training sample counts match the published figures of 8,948 and 17,896 exactly, confirming that the data pipeline reconstructs the original protocol rather than approximating it.

Each detected slot is warped to a uniform 120×46 pixels by perspective transform. Augmentation combines horizontal flip and HSV saturation and exposure jitter in the range $[1.0, 1.5]$, each applied with probability 0.5. The classifier is initialized from ImageNet weights and optimized with Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$) at an initial learning rate of 10^{-4}, decayed by a factor of ten every 45,000 gradient steps, at batch size 32.

Table IV reports the resulting trajectory over 323 epochs.

TABLE IV
VPS-NET OCCUPANCY CLASSIFIER, PS2.0 (SELECTED EPOCHS)

Epoch	Train Loss	Train Acc	Val Acc
1	0.1672	94.52	97.24
10	0.0591	98.19	98.34
33	0.0265	99.13	**98.53**
60	0.0116	99.63	98.53
100	0.0085	99.72	98.25
200	0.0004	99.99	98.02
323	0.0001	100.00	98.07

Peak validation accuracy of 98.53% is reached at epoch 33 and is not subsequently improved upon across a further 290 epochs. Training accuracy meanwhile continues to rise to 100.00% and training loss falls to 5×10^{-5}, while validation accuracy drifts down to 98.07%. This is textbook overfitting past the optimum, and it anticipates at larger scale a pattern that recurs in the occupancy classifier of Section VI: weights must be selected by best validation accuracy rather than by final epoch. The epoch 33 checkpoint is the one exported.

The reproduced 98.53% sits below the 99.63% precision and 99.31% recall reported originally [6], which trained detection and classification jointly where only the classification stage was retrained here; given the exact match in sample counts, the gap reflects that difference in scope rather than a defect in the reproduction. The weights are packaged in the inference service alongside the occupancy classifier and selectable through the same API, but automated geometry extraction is not yet wired into registration.

G. YOLOv8 Reproduction Study

In parallel with the production classifier, a reproduction study of a recent detection-based approach was undertaken to determine whether the detection formulation would justify its additional cost. The target was the five-variant YOLOv8 backbone comparison of Pokhrel and Dao [7], which evaluates a stock CSPDarknet53 baseline against ResNet-18, VGG-16, EfficientNetV2-S, and Ghost-P2 backbones on PKLot.

A self-contained reproduction harness was constructed, comprising the four custom backbone configurations and an evaluation notebook that reports test-split and validation-split metrics side by side and diffs them against the published tables. Two defects in the original artifacts were identified and verified locally:

- The published YAML configurations omit the `scales:` block, causing Ultralytics to build the PANet neck at depth = width = 1.0 rather than $0.33/0.25$ — a model roughly four times larger than the paper's complexity table describes. Restoring the block reproduces that table to four significant figures.
- The published accuracy table reports EfficientNetV2 as best by mAP50:95, while the authors' own logs record VGG-16 as superior (0.98613 against 0.98269), indicating the ranking conclusion is unverified.

The full sweep requires 8–10 hours on a T4 and was not completed within the semester; the harness, configurations, and verified discrepancy analysis are this study's deliverables. Its practical conclusion is that detection offers no accuracy advantage sufficient to justify its cost under fixed-camera deployment, supporting the classification approach adopted in production.

VI. AI/ML RESULTS

A. Phase 1 — Head Training

Table V reports per-epoch results. Validation accuracy peaks at 97.28% in epoch 5.

TABLE V
PHASE 1 RESULTS (VALIDATION ON HELD-OUT LOT UFPR05)

Epoch	Train Loss	Train Acc	Val Loss	Val Acc
1	0.0466	0.9848	0.1094	0.9631
2	0.0421	0.9862	0.0916	0.9698
3	0.0423	0.9861	0.1057	0.9655
4	0.0414	0.9864	0.1178	0.9624
5	0.0416	0.9866	0.0803	**0.9728**

Validation accuracy is non-monotonic across epochs 2–4 while training accuracy stays flat near 98.6%. With the

backbone frozen the head has limited capacity to close the domain gap to the unseen lot, so the fluctuation reflects that ceiling rather than overfitting.

B. Phase 2 — Full Fine-Tuning

Table VI reports the fine-tuning phase. Validation accuracy peaks at 99.18% in epoch 9.

TABLE VI
PHASE 2 RESULTS (VALIDATION ON HELD-OUT LOT UFPR05)

Epoch	Train Loss	Train Acc	Val Loss	Val Acc
1	0.0134	0.9966	0.0560	0.9816
2	0.0074	0.9984	0.0253	0.9915
4	0.0046	0.9990	0.0285	0.9910
6	0.0034	0.9992	0.0374	0.9886
7	0.0032	0.9992	0.0868	0.9729
9	0.0024	0.9993	0.0306	**0.9918**
10	0.0022	0.9994	0.0378	0.9893

Unfreezing the backbone improves accuracy on the held-out lot by 1.90 percentage points over Phase 1, from 0.9728 to 0.9918. Equivalently, the residual error rate falls from 2.72% to 0.82%, a relative reduction of approximately 70%. Because training accuracy was already 98.7% before Phase 2 began, this gain is attributable to improved cross-lot generalization rather than to additional fitting of the training distribution.

The epoch 7 excursion to 0.9729, against a training accuracy of 0.9992, indicates that individual optimization steps can transiently misalign the representation with the unseen lot. Selecting weights by best validation accuracy rather than final epoch is therefore a necessary part of the protocol; epoch 9 weights are exported for deployment.

C. Positioning Against Prior Work

Custom CNNs trained from scratch on this data are reported near 93% under same-lot evaluation; the present model reaches 97.28% after Phase 1 and 99.18% after Phase 2 under the stricter cross-lot protocol.

That figure is nonetheless measured on an unseen *international* lot, not on Bangladeshi footage. Prova et al. [9] establish that models trained only on international data transfer to Bangladeshi conditions at 86–90%. The honest interpretation is that this model exceeds prior work on the international benchmark and is expected to exceed the 86–90% transfer baseline in Dhaka, but that the expectation is untested until local footage is collected. Section X treats this as the system's principal open risk.

D. Deployment Behaviour

The exported Phase 2 weights are served by a FastAPI service exposing single-image and batch prediction endpoints. A request to `/api/v1/inference/predict` for a given slot returns the predicted label, a confidence score, and an inference identifier — for example `empty` at confidence 0.8796 — and each result is persisted to MongoDB for audit and history retrieval.

The consuming backend applies a rule that is asymmetric by design: a slot is marked unavailable only when the model predicts *occupied* with confidence ≥ 0.90, and all other outcomes resolve to *available*. The rationale is that reservation state, not visual inference, is authoritative for whether a driver can use a slot, so a low-confidence prediction should not suppress a slot the booking layer would permit. The cost is that a driver may occasionally arrive at a facility whose true occupancy exceeds the reported figure.

VII. BACKEND PLATFORM

A. Architecture

The platform is organized as two cooperating services with three client surfaces (Fig. 1): a NestJS application owning identity, facilities, reservations, and orchestration, and a FastAPI service owning model execution. Clients communicate only with the former.

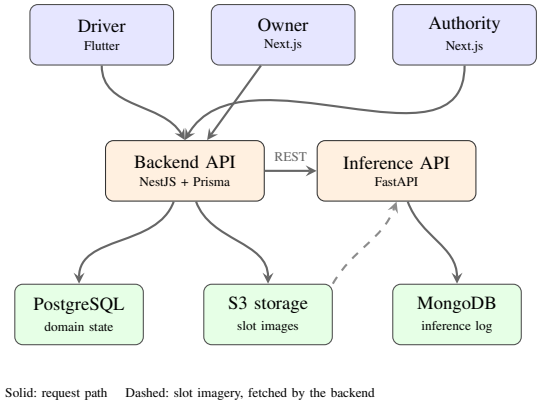

Fig. 1. CityPulse service architecture as implemented.

The implemented stack comprises Flutter for the driver application; Next.js for the owner portal and authority command centre; NestJS with Prisma for authentication, facilities, bookings, and scheduling; FastAPI serving PyTorch models for inference; PostgreSQL for domain state, MongoDB for inference history, and S3-compatible object storage for imagery; with contracts published as OpenAPI. Storage and messaging technologies considered during design but not implemented are discussed in Section X.

B. Data Model

The PostgreSQL schema comprises twenty-one models and eleven enumerated types under Prisma management. Core entities are `users` with role-specific profile tables; `parking_spaces` and their constituent `parking_slots`; `cameras` carrying each facility's RTSP endpoint and setup status; `space_photos`; and `bookings`. Supporting tables cover roles, permissions, sessions, OAuth accounts, pricing rules, payment transactions, notifications, and a four-level Bangladeshi administrative hierarchy of divisions, districts, thanas, and areas.

C. API Surface

Table VII lists the implemented endpoints. All routes require a bearer token except those explicitly marked public, and all are published as an OpenAPI document served at `/docs`.

TABLE VII
IMPLEMENTED REST ENDPOINTS

Group	Endpoints
`/auth`	`register`, `login`, `refresh`, `logout`, `sessions` (list, revoke)
`/spaces`	create, list, `nearby` (public geospatial search), detail with slot status, update, delete, `:id/infer`, `:id/photos` (upload, presign, delete)
`/bookings`	create, driver history, detail, `:id/cancel`
`/profiles`	`me` (read, update)
`/users`	list, detail, update
`/admin`	roles and permissions CRUD, user role assignment, role–permission binding

D. Inference Pipeline

Occupancy state is refreshed by a scheduled service within the NestJS application on a default ten-minute interval, configurable through the `INFERENCE_INTERVAL_MS` environment variable. For every registered slot image, each cycle generates a presigned S3 URL, retrieves the image bytes under a 60-second timeout, forwards them to the FastAPI inference endpoint under a 90-second timeout, applies the confidence rule of Section VI, and persists the resulting status, label, and confidence against the slot record.

Both network stages are bounded by explicit abort signals, so an unreachable storage endpoint or stalled inference request cannot block a cycle indefinitely. Records carry an update timestamp, and the nearby-search response marks any facility whose slot data exceeds the staleness threshold, letting clients distinguish current data from data that could not be refreshed.

E. Availability Reconciliation

The system maintains two independent notions of a slot being unusable: it is physically occupied according to visual inference, or it is held by an active reservation. The backend resolves these into one figure using the disjunction

$$\text{unavailable} = \text{booked}_{\text{now}} \ \vee \ \text{occupied}_{\text{visual}}.$$

A slot with an active booking overlapping the present instant is therefore reported unavailable regardless of what the camera shows, which prevents a reserved slot from being offered to a second driver during the interval between reservation and arrival. The response additionally exposes a per-slot label distinguishing *booked* from *occupied* from *empty*, so that an owner can observe when physical occupancy and reservation state disagree — the signal that would indicate unauthorized use of a reserved slot.

F. Search and Access Control

Nearby search accepts a coordinate pair and radius, applies a bounding-box prefilter in the database query, computes great-circle distance per candidate, and returns results filtered to the radius and sorted by proximity, each carrying slot counts, distance, and the staleness flag.

Authentication issues a signed access token with a refresh token whose hash is persisted as a revocable session record. Route access is mediated by a JWT guard and a permissions guard backed by the roles and permissions tables, with a public decorator marking the few unauthenticated routes. Owners, drivers, and authorities consequently reach disjoint portions of the API surface.

G. Designed but Unimplemented Domain Surface

The data model deliberately anticipates functionality beyond the present service layer. Eight of the twenty-one schema models have no corresponding module, representing design work completed during Sprint 2 that the semester did not reach. Table VIII summarizes them.

TABLE VIII
MODELLED ENTITIES WITHOUT A SERVICE LAYER

Entity	Design intent
`payment_transactions`	Settlement over bKash, Nagad, card, or cash, with gateway reference, metadata, and refund fields
`pricing_rules`	Surge pricing: occupancy threshold (default 80%), factor in basis points (default $1.5\times$), floor and ceiling rates
`notifications`	Typed notifications with read and sent timestamps: booking confirmed or cancelled, slot available, payment receipt, system alert
`user_oauth_accounts`	Federated sign-in via Google, Facebook, or Apple
`divisions` → `areas`	Four-level administrative hierarchy for jurisdiction-scoped authority views

Two enumerations extend this to the local regulatory context: `licence_type` distinguishes professional from non-professional licences, and `vehicle_class` encodes the BRTA classes — heavy, medium, light, motorcycle, three-wheeler, and public service vehicle — against which a driver may hold multiple entitlements. Neither is enforced at booking time. Consequently the unbound portal views of Section VIII are each blocked on a specific missing endpoint rather than on interface work: revenue history on `payment_transactions`, zone management on the geographic hierarchy, and the camera view on camera lifecycle endpoints beyond the registration-time write.

VIII. CLIENT APPLICATIONS

A. Web Portal

The Next.js portal serves owners and authorities from a single application with role-based routing. Authentication and data access are mediated through a repository abstraction over a shared HTTP client, keeping transport and token-storage concerns isolated from page components; separate repositories cover authentication, profiles, and facilities. Unit tests cover the API client, session handling, routing destination

resolution, and phone normalization. Route entry files delegate to named implementation components, so the routing surface stays readable independently of page logic.

The portal divides into two tiers. The owner dashboard and the facility registration flow are bound to live backend data through the repositories above, covering the complete onboarding path from account creation to a registered facility with uploaded imagery and camera metadata. A second tier — authority analytics, enforcement logs, zone management, owner revenue history, and the camera management view — is implemented as working interface at 150 to 200 lines each, with layout, state, and interaction complete, but is not yet bound to live data because the corresponding backend endpoints do not exist. Section VII-G identifies those endpoints.

B. Mobile Application

The Flutter application targets drivers. On launch it acquires the device location and queries `/spaces/nearby`, rendering results as an interactive Google Map with per-facility markers and as a distance-sorted list, with client-side search filtering the retrieved set. Authentication, bookings, and profile management are backed by the live API using bearer tokens from a session service that persists credentials between launches. The booking flow supports creation, detail inspection, cancellation, and history with local removal of dismissed records, and prices render in Bangladeshi Taka.

Route guidance hands the selected facility's coordinates to Google Maps through a directions deep link rather than rendering turn-by-turn instructions in-application, avoiding a routing API dependency and its key management at the price of a context switch.

IX. System Integration and Evaluation

The prototype was exercised against implemented workflows rather than under production load. Verified end to end were owner registration and facility onboarding with imagery and camera metadata; scheduled inference updating slot occupancy with counts reflected in both clients; reservation creation, cancellation with history retention, and driver-scoped listing; preservation of unavailability across an active booking irrespective of visual predictions; distance-ordered geospatial search; role separation across all three credential types; and reproduction of both training pipelines through to exported weights. Load testing, latency measurement, and field deployment against live cameras were not performed (Section X).

X. Limitations

We state the boundary of the present system explicitly.

- **No local validation data.** The 99.18% figure is measured on an unseen international lot. No Bangladeshi parking imagery has been collected or annotated, so performance under Dhaka conditions is inferred from [9] rather than measured. This is the most significant open risk in the system.
- **Still images, not video streams.** Camera RTSP endpoints are captured at registration and stored, but no stream is consumed. The inference pipeline operates on still images retrieved from object storage. Continuous stream ingestion remains to be implemented.
- **Polled updates, not pushed.** Clients obtain availability by request, and the ten-minute refresh interval bounds staleness. Push delivery, a cache tier, and a message broker were specified during design but are not implemented.
- **Partial service layer.** Eight modelled entities have no module (Section VII-G), leaving five portal views unbound to live data and no payment, notification, or pricing capability.
- **Manual slot annotation.** Slot polygons are defined per camera at onboarding; VPS-Net is trained and packaged but not wired into registration.
- **Asymmetric confidence rule.** Occupancy is asserted only above 0.90 confidence, so the system under-reports rather than over-reports occupancy. The effect on driver experience is unmeasured.
- **Surveillance features not implemented.** Bangla plate recognition and congestion detection were surveyed only.

XI. Project Timeline

The project followed an explore-then-build cadence across six two-week sprints, shown in Fig. 2.

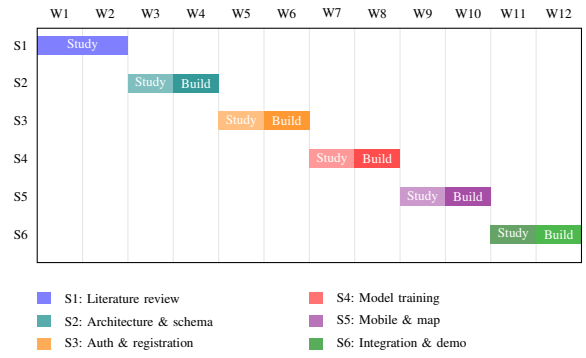

Fig. 2. Project timeline across twelve weeks.

Each sprint paired an exploration week with a build week, as the figure's legend indicates; field observation of Dhaka parking facilities was conducted during Sprint 1.

XII. Future Work

Future work falls into completing what is designed but unbuilt, extending what is built, and the research direction beyond parking.

A. Machine Learning

The immediate priority is collecting and annotating Bangladeshi parking imagery to enable a third fine-tuning phase, converting the transfer estimate of Section VI into a measured figure. Wiring the trained VPS-Net model of Section V-F into facility registration would automate slot geometry extraction and remove the manual annotation bottleneck inherited from [9]. Deploying the exported weights to a Raspberry Pi at the parking site would move inference to the

edge, eliminating the image upload round trip and permitting a refresh interval far below the present ten minutes. Both models also warrant a fuller evaluation: precision, recall, and per-class confusion on the held-out lot, and measured inference latency on the target hardware, none of which the current results report.

B. Backend Services

Each entity in Table VIII corresponds to a service that would unblock an interface already built. Payment settlement over bKash and Nagad is the highest-value addition, closing the loop between reservation and revenue and activating the owner revenue view; since the schema already carries gateway reference, metadata, and refund fields, the work is a module and gateway integration rather than a data model change. Notification delivery would activate the `slot_available` alert that drives retention, and is the natural point to introduce push delivery, the cache tier, and the message broker together, since all three share the same infrastructure. Demand-based pricing can then run against the surge threshold already modelled, and enforcing BRTA vehicle class at booking would prevent a driver reserving a slot their licence does not cover. Continuous RTSP ingestion would replace the still-image pipeline. Operationally, the system needs load testing, latency measurement, and a deployment pipeline before any field trial.

C. Web Portal and Mobile

The five unbound portal views need repository bindings once their endpoints exist; because the interface tier is complete, that work is data wiring rather than redesign. Beyond it, the command centre would benefit from live map-based occupancy visualization and the owner dashboard from utilization charts drawn from the inference history. On mobile, in-app turn-by-turn guidance would remove the context switch to Google Maps.

D. Research Direction

Beyond parking, the surveyed groundwork on Bangla plate recognition and congestion detection sets the direction for the next phase, as does occupancy forecasting — for which the inference history already persisted in MongoDB constitutes the training corpus.

XIII. Conclusion

This report has presented CityPulse, a parking availability platform built around automated vision-based occupancy detection. The central result is a MobileNetV2 classifier attaining 99.18% accuracy on a facility withheld entirely from training, obtained through two-phase transfer learning on a merged 808,991-image corpus under a cross-lot protocol chosen to measure generalization to newly registered facilities rather than to a familiar camera.

That model is operational rather than academic: served through a documented inference API, invoked on a schedule by a backend that reconciles its predictions with reservation state, and consumed by a Flutter application and a Next.js portal over live REST contracts. A second model, VPS-Net, was reproduced to 98.53% on ps2.0 as a route toward removing the manual per-camera annotation that prior work identifies as the main barrier to scaling such systems.

The boundaries are equally explicit. No Bangladeshi imagery has been collected, so deployment performance is inferred from [9] rather than measured; camera streams are registered but not consumed; eight modelled entities await a service layer; and the surveillance capabilities motivating the platform's direction have been surveyed but not built. These are stated as limitations rather than deferred features because the distinction matters to anyone assessing what the prototype demonstrates.

What the work establishes is that the core premise holds: occupancy can be determined automatically, at accuracy sufficient for the application, using camera hardware facilities already possess, and delivered to a driver's phone through a working system.

References

[1] World Bank, *Dhaka: Improving Living Conditions in the City*. Washington, DC: World Bank Group, 2018. https://www.worldbank.org/en/country/bangladesh

[2] Bangladesh Telecommunication Regulatory Commission, *Mobile Phone Subscribers Statistics*, 2024. https://www.btrc.gov.bd

[3] P. R. L. de Almeida, L. S. Oliveira, A. S. Britto Jr., E. J. Silva Jr., and A. L. Koerich, "PKLot — A robust dataset for parking lot classification," *Expert Systems with Applications*, vol. 42, no. 11, pp. 4937–4949, 2015.

[4] G. Amato, F. Carrara, F. Falchi, C. Gennaro, C. Meghini, and C. Vairo, "Deep learning for decentralized parking lot occupancy detection," *Expert Systems with Applications*, vol. 72, pp. 327–334, 2017.

[5] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.

[6] W. Li, L. Cao, L. Yan, C. Li, X. Feng, and P. Zhao, "Vacant parking slot detection in the around view image based on deep learning," *Sensors*, vol. 20, no. 7, p. 2138, 2020. DOI: 10.3390/s20072138

[7] A. Pokhrel and G. Dao, "Optimizing YOLOv8 for parking space detection: Comparative analysis of custom backbone architectures," *arXiv preprint arXiv:2505.17364*, 2025.

[8] Y. Yuldashev, M. Mukhiddinov, A. B. Abdusalomov, R. Nasimov, and J. Cho, "Parking lot occupancy detection with improved MobileNetV3," *Sensors*, vol. 23, no. 17, p. 7642, 2023. DOI: 10.3390/s23177642

[9] R. R. Prova, T. Shinha, A. B. Pew, and R. M. Rahman, "A real-time parking space occupancy detection using deep learning model," in *Proc. IEEE International IOT, Electronics and Mechatronics Conference (IEMTRONICS)*, 2022. DOI: 10.1109/IEMTRONICS55184.2022.9795771

[10] N. Hasin, M. A. R. Nishat, M. Islam, K. S. Al Hasan, and A. Newaz, "A robust deep learning framework for Bangla license plate recognition using YOLO and vision-language OCR," *arXiv preprint arXiv:2603.10267*, 2025.

[11] A. Ashrafee, A. M. Khan, M. S. Irbaz, and M. A. Al Nasim, "Real-time Bangla license plate recognition system for low-resource video-based applications," *arXiv preprint arXiv:2108.08339*, 2021.

[12] K. A. Azam, H. Masum, M. Rahaman, and A. B. M. A. A. Islam, "Towards intelligent traffic signaling in Dhaka city based on vehicle detection and congestion optimization," *arXiv preprint arXiv:2510.16622*, 2025.

[13] ParkWhiz Inc., *ParkWhiz: Find and Book Parking*, 2023. parkwhiz.com

[14] Dhaka North City Corporation, "Smart parking: DNCC debuts app in Gulshan," *Press Xpress*, Nov. 2023. pressxpress.org

[15] R. Rahman, "Building a parking solution for Bangladesh," *Future Startup*, Jul. 2019. futurestartup.com